

MAYO 2024

Proyecto final de grado

BOOK MARKED

Aplicación para la gestión de tus lecturas y opinión sobre ellas

REALIZADO POR:

Isabel del Carmen Romero Ruiz

TUTOR DEL PROYECTO:

Rafael Perez Corro

CENTRO:

Colegio Santísima Trinidad



BookMarked

Índice

1. Estudio del problema y análisis del sistema.....	2
1.1. Introducción.....	2
1.2. Finalidad.....	3
1.3. Objetivos.....	3
2. Modelado de la solución.....	3
2.1. Recursos humanos.....	3
2.2. Recursos de hardware.....	3
2.3. Recursos software.....	4
3. Planificación.....	4
3.1. Diseño del proyecto.....	4
3.2. Diseño de las vistas.....	5
3.3. Flujo de las vistas.....	10
3.4. Branding aplicación.....	11
4. Desarrollo de las vistas.....	11
4.1. Clase book (código en Anexo 6).....	11
4.2. ContentView (Código en Anexo 1):.....	12
4.3. DetailView (Código en Anexo 2):.....	13
4.4. AddBookView (Código en Anexo 3):.....	14
4.5. EmojiRatingView (código de Anexo 5):.....	15
4.6. RatingView (código en Anexo 4):.....	16
4.7. ScreenLaunch o SplashScreen.....	17
4.8. Vista de la aplicación en modo oscuro.....	18
5. Uso de SwiftData.....	19
6. Problemas que han surgido y soluciones.....	19
7. Fase de pruebas.....	20
8. Conclusión.....	20
9. Documentación del sistema desarrollado.....	21
10. Bibliografía.....	22

1. Estudio del problema y análisis del sistema.

1.1. Introducción.

Este proyecto consiste en crear una aplicación que nos sirva para llevar todas nuestras lecturas siempre encima, tendremos acceso a una lista personal con nuestras lecturas y nuestra opinión sobre ellas, será totalmente personal y no tendrá acceso ninguna otra persona, esta lista estará ordenada y clasificada en distintos grupos dependiendo de la puntuación que le hayamos dado a ese libro, también podremos eliminar lecturas de la lista y visualizar cuántos libros hemos leído.

¿Cómo surge esta idea?

Esta idea surge de la necesidad de tener un acceso rápido y personal (al que nadie más tiene acceso) a tu lista de libros leídos en la que tú puedas escribir tu reseña sincera con total libertad de ese libro sin miedo a que otras personas te juzguen porque no te haya gustado ese libro tan famoso que a todo el mundo le encanta.

1.2. Finalidad.

La finalidad de esta aplicación es poder llevar siempre nuestros libros leídos encima, no es la primera vez que te encuentras con un amigo lector que te pregunta tu opinión de un libro y ya ni te acuerdas, con esta aplicación siempre tendrás ahí tus libros valorados y tu opinión sobre ellos.

1.3. Objetivos.

Los servicios que ofrece este proyecto es que añadas tus lecturas (el libro que hayas terminado de leer o el que estés leyendo actualmente) junto con tu opinión y valoración de ese libro, así siempre podrás ver qué te pareció esa historia y se lo podrás recomendar a quién te pregunte basándote en lo que tengas apuntado en tu aplicación.

2. Modelado de la solución.

2.1. Recursos humanos.

Para realizar este proyecto necesitamos de un desarrollador de software de iOS (preferiblemente un desarrollador de SwiftUI), y esa misma persona se encargará también del desarrollo del diseño y la interfaz de usuario con programas como Figma o Canva.

2.2. Recursos de hardware.

Para poder desarrollar este proyecto necesitamos un ordenador Mac con la última actualización del sistema operativo MacOS con el que podamos desarrollar aplicaciones para la última versión de iOS 17, además en este ordenador tendremos que poder programar en SwiftUI.

Esta aplicación estará orientada a dispositivos móviles con sistema operativo iOS, sino, móviles iPhone.

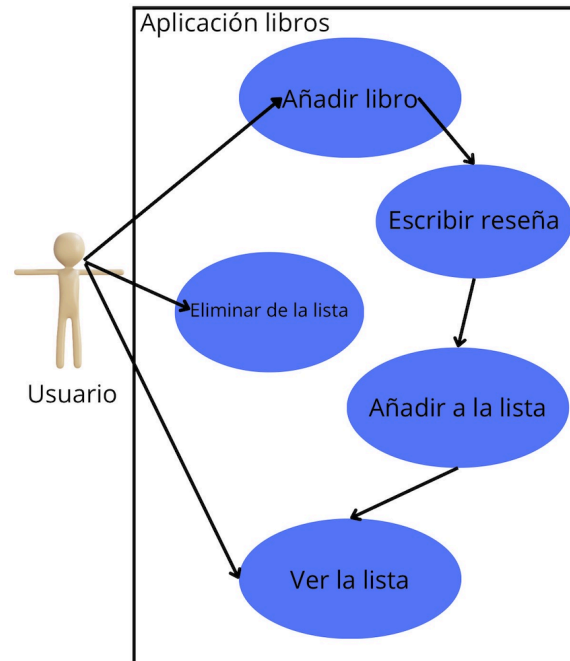
2.3. Recursos software.

Para poder programar esta aplicación necesitaremos el entorno de desarrollo integrado (IDE) XCode con el que podremos desarrollar aplicaciones para Mac, iPhone, iPad, Apple Watch, Apple TV y la última novedad de la familia Apple, las vision Pro. En XCode es el único entorno en el que actualmente podemos desarrollar el lenguaje SwiftUI que es el lenguaje que utiliza iPhone.

3. Planificación.

3.1. Diseño del proyecto.

A la hora de diseñar el proyecto hice el *diagrama de usos* para hacerme una idea de cómo va a interactuar el usuario con la aplicación, el usuario al acceder a la aplicación podrá añadir un libro a su lista, ver su lista o eliminar un libro de la lista, si elige añadir un libro a la lista entonces tendrá que escribir una reseña y después añadir el libro a la lista



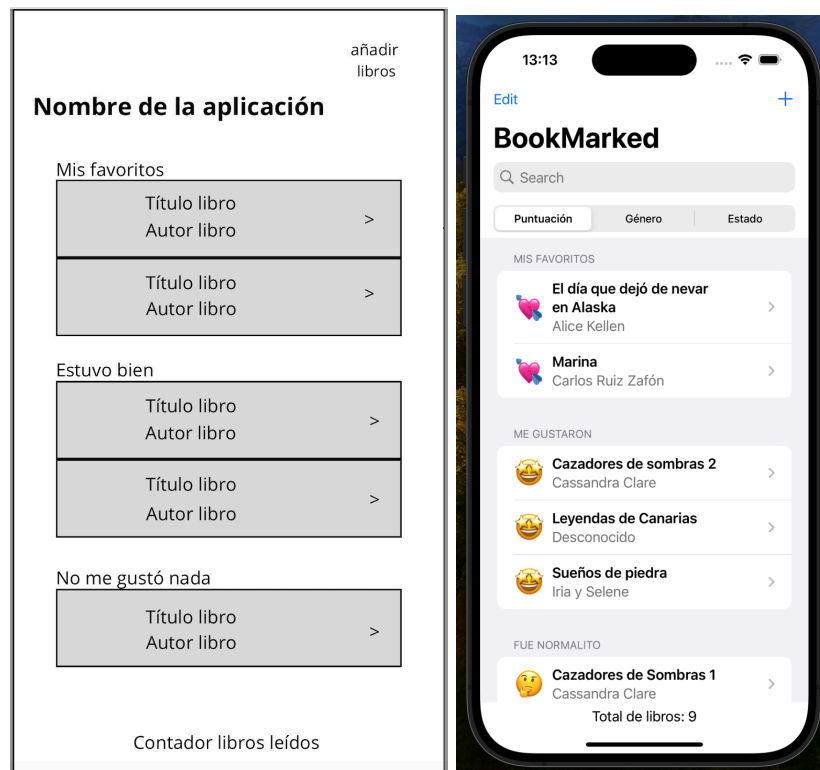
No realicé un diagrama de clases porque en mi aplicación sólo tengo una clase, que es la clase libro y con una única clase no tenía sentido el hacer el diagrama de clases.

3.2. Diseño de las vistas.

Antes de ponerme a programar hice el diseño de las vistas para saber cómo quería que quedase visualmente la aplicación.

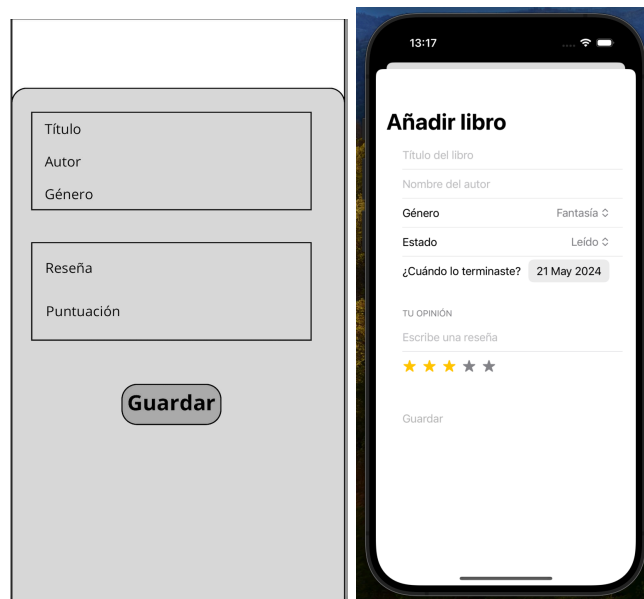
ContentView o vista principal: aquí podremos ver nuestra lista con las lecturas divididas en grupos, aquí también tendremos un botón con el que podremos añadir libros y al final de esta vista tendré un contador de libros.

Vista diseñada y vista final



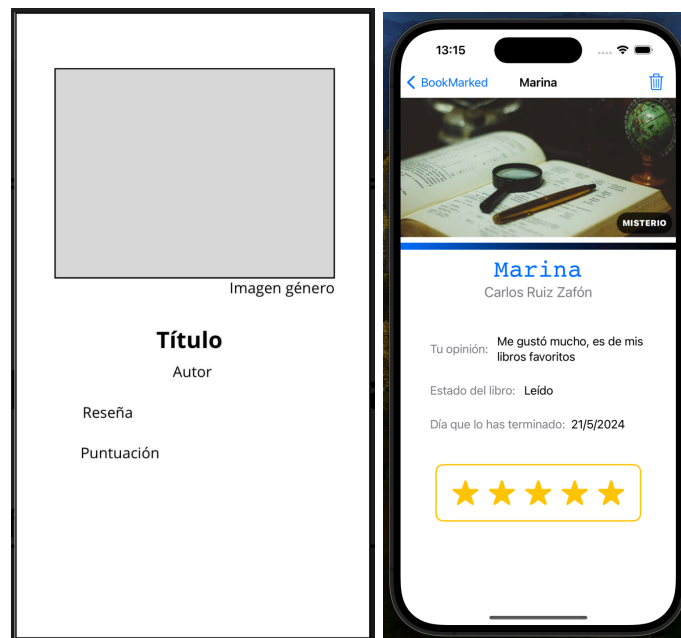
AddBookView: la vista que rellenaremos cuando queramos añadir un nuevo libro, será un formulario con un botón de guardar.

Vista diseñada y vista final:

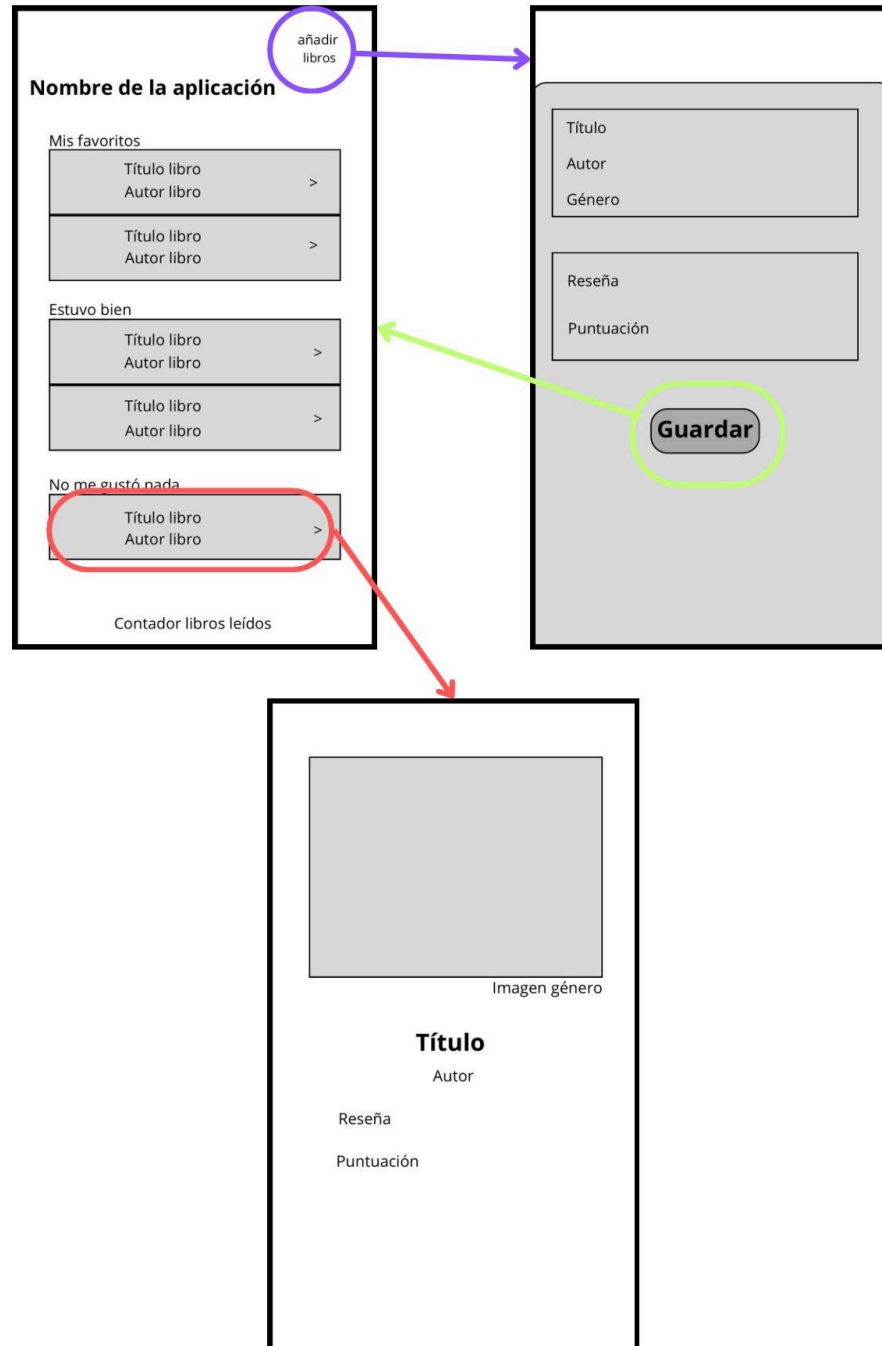


DetailView: la vista a la que accederemos cuando queramos ver los detalles de una lectura, aquí tendremos una imagen dependiendo del género que sea ese libro, el título, el autor, la reseña y la puntuación, en esta vista sólo podremos visualizar lo que ya hayamos escrito, no podremos modificar nada.

Vista diseñada y vista final:



3.3.Flujo de las vistas.



3.4. Branding aplicación.

Quería que la aplicación tuviese un icono propio, fácil de reconocer y que se quedase en la memoria de los usuarios, para ello he utilizado las dos letras iniciales del nombre de la aplicación que es “BookMarked” y las dos letras son B y M, he utilizado dos letras que parecen escritas sobre papel o utilizando como herramienta el papel para darle ese toque de los libros físicos, y de fondo del icono podemos ver un degradado de azul pastel porque es un color que transmite paz y tranquilidad.



4. Desarrollo de las vistas.

Para el correcto desarrollo y funcionamiento de esta app he creado un total de 5 vistas compuestas por: *ContentView*, *DetailView*, *EmojiRatingView*, *AddBookView*, *RatingView* y la *LaunchScreen* o *SplashScreen*

Cada una de estas vistas tienen las siguientes funcionalidades:

4.1. Clase book (código en Anexo 6):

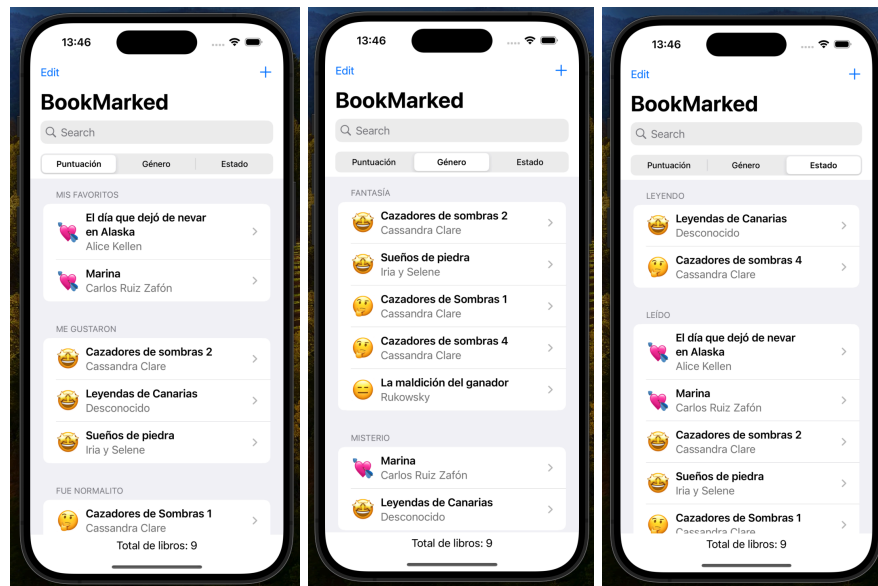
Lo primero que hacemos es crear esta clase ‘book’ donde tendremos todas las propiedades almacenadas con SwiftData (en un punto más adelante explico qué es SwiftData y cómo funciona), en esta clase guardaremos el título, autor, género, reseña, puntuación, día actual. En esta clase también añadí una función que convierte el Date en un String para que lo podamos visualizar sin problemas en el *DetailView*.

4.2. *ContentView* (Código en Anexo 1):

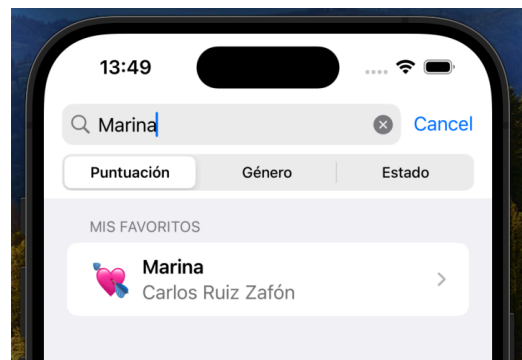
Esta es la vista principal y la vista a la que accedemos según entramos a la app, aquí podremos ver toda nuestra lista de libros clasificados en distintas secciones: Puntuación, Género y Estado; al ver los libros filtrados por puntuación lo que visualizamos es nuestra lista de lecturas clasificadas según la puntuación que le hayamos dado diferenciadas en distintos grupos: “ Mis favoritos” (5/5), “Me gustaron”(4/5), “Fue normalito”(3/5), “Meh”(2/5) y “He perdido el tiempo”(1/5), y dependiendo de esta puntuación tendremos un emoji que representa a cada uno de los grupos; si queremos visualizar los libros filtrados por el género visualizamos la lista de las lecturas dividida en los distintos géneros: fantasía, infantil, misterio, poemario, romance y thriller; y por último si queremos ver la lista filtrada según el estado de los libros veremos las lecturas divididas en leídos (que son los libros que ya hemos finalizado) y leyendo (que son nuestras lecturas actuales, gracias a esto podremos ir actualizando nuestra opinión según leemos el libro).

En esta vista también podemos ver el nombre de la aplicación en la parte superior “BookMarked” y en la parte superior también tenemos el buscador de libros (en el que podemos localizar los libros a partir de su título o nombre del autor)y el picker del filtrado que expliqué antes, además vemos un símbolo ‘+’ en el que si clicamos podremos añadir nuestra nueva lectura; en la parte inferior de esta primera vista podremos visualizar el número de libros leídos, además en esta vista tendremos la opción de eliminar las lecturas deslizando el dedo sobre los títulos de los libros o clicando sobre “edit”. También en la parte superior vemos un símbolo + en el que si pulsamos accedemos a la vista AddNewBook. Un pequeño detalle que añadí es que todas las listas, dando igual cómo se filtren, cada una de las secciones estará ordenada mostrando primero los libros con mayor puntuación y por último los que tienen la puntuación más baja.

La vista con cada uno de los filtros:



Buscador de libros:

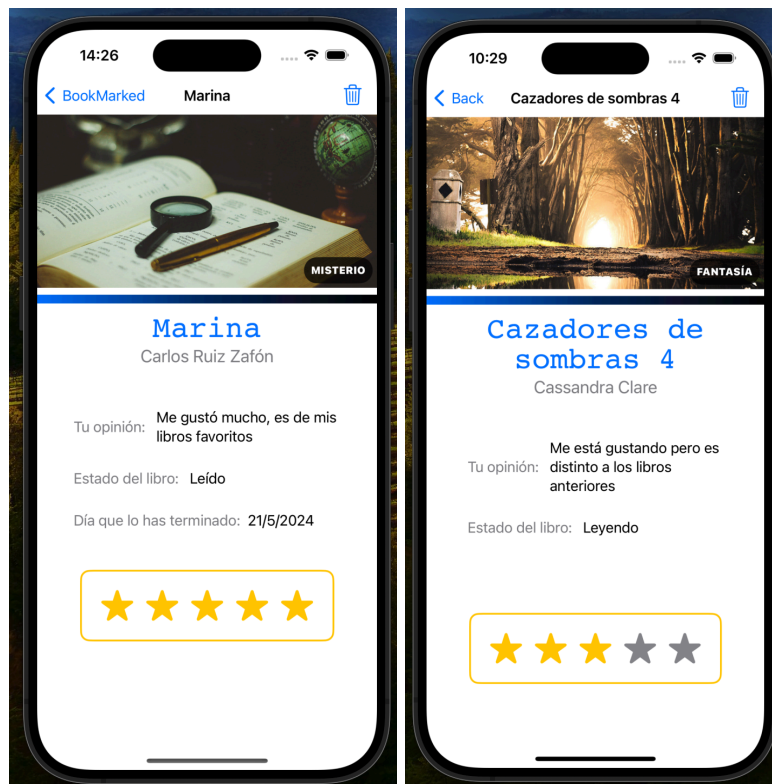


Para que esta vista se vea así lo que he hecho es que toda la vista sea una lista dividida en secciones y cada una de estas secciones tiene que cumplir un *if* conforme si están vacías no aparecen y si tienen información sí aparecen, dentro de cada sección llamamos al emoji que le corresponda según la puntuación que tenga, y también llamamos al título y autor del libro que es la información que se ve de cada libro que forma parte de la lista, además añadí un modificador *‘.onDelete’* a cada una de las secciones que llama a la función *‘deleteBooks’* para que cuando deslicemos sobre los libros podamos eliminarlo de la lista.

4.3. *DetailView* (Código en Anexo 2):

A esta vista accederemos cuando pulsemos sobre alguno de los libros que tenemos en la lista, aquí lo primero que veremos es una imagen correspondiente al género al que pertenezca esta lectura, a continuación podemos leer el título del libro, su autor, nuestra opinión, el estado del libro si lo estamos leyendo o ya lo hemos terminado, en caso de que el estado del libro sea “Leído” también veremos el día que hayamos terminado el libro, pero en caso de que el estado del libro sea “Leyendo” no visualizamos ninguna fecha (es un detalle muy pequeño pero que es importante para mejorar la experiencia de usuario), después de esto podemos ver en estrellas la puntuación que le has dado a la lectura sobre 5.

Un ejemplo de cómo se ve el *DetailView* de un libro leído (izquierda) y de otro que se está leyendo (derecha):



Para que esta vista funcione correctamente lo primero que tengo en la estructura es un *ZStack* en el que llamo a la imagen que le corresponda según el género que seleccionamos en *DetailView*, además sobre esta imagen aparece el nombre del

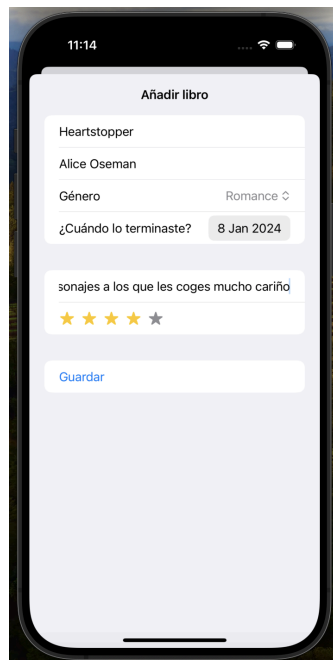
género del libro; a continuación de esto tengo distintos Text donde llamo a las distintas propiedades que hemos guardado (el título, autor, opinión, estado del libro y el día que lo terminaste); en la parte superior de la pantalla podemos ver el título del libro, también en la parte superior podemos ver el símbolo de una papelera), si clicamos sobre esta imagen nos aparece una alerta advirtiendonos si estamos seguros que queremos eliminar este libro.

4.4. *AddBookView* (Código en Anexo 3):

En esta vista lo que tenemos es un formulario que no te dejará guardar la información hasta que no hayas rellenado todos sus campos, primero tendremos que rellenar el campo del título del libro, el nombre del autor, el género que le corresponde (y dependiendo de esto en la vista *DetailView* tendremos una imagen u otra), el estado del libro si lo estamos leyendo o ya lo hemos leído; si es un libro que ya hemos leído entonces podremos elegir el día que se finalizó la lectura, en cambio si es un libro que estamos leyendo no podremos seleccionar ninguna fecha , a continuación escribiremos una reseña con nuestra opinión sincera ya que será personal y nadie tendrá acceso a ella, la puntuación sobre 5 estrellas y el botón de guardar.



Y si rellenamos todos los campos vemos que el botón de guardar se activa.

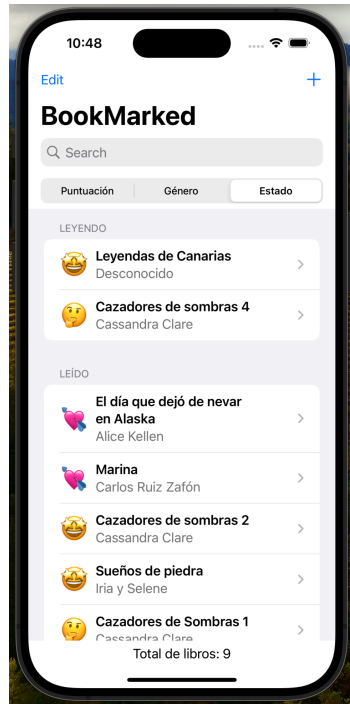


Para que esta vista funcione correctamente primero declaramos las propiedades que necesitamos, hacemos la validación del formulario y creamos los géneros que queramos utilizar; en el cuerpo de la vista creamos un formulario que dividimos en secciones y tendremos los campos para el título, autor, un datePicker en el que podremos seleccionar en un calendario el día que terminamos el libro (en este calendario el último día que podremos seleccionar será el día actual), la siguiente sección será para nuestra opinión en la que escribiremos nuestra reseña y le daremos una puntuación al libro y por último tenemos el botón de guardar que sólo se activará si si tenemos todos los campos obligatorios rellenos.

4.5. *EmojiRatingView* (código de Anexo 5):

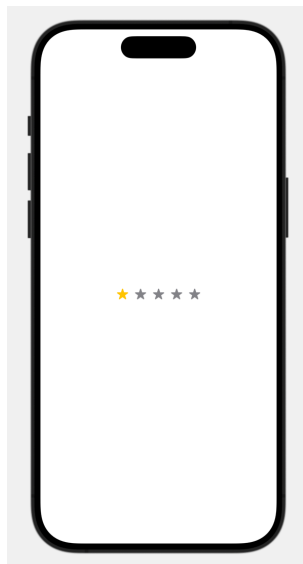
Aquí lo único que hacemos es adjudicar un emoji a cada una de las puntuaciones para que este emoji se visualice en la página principal, para esto usamos un switch en el que en caso de tener 1 estrella el emoji será: 😞, si la puntuación es 2: 😐, si es 3: 😊, si es 4: 😄 y si es 5/5: 💖.

Vista de los emojis en ContentView



4.6. *RatingView* (código en Anexo 4):

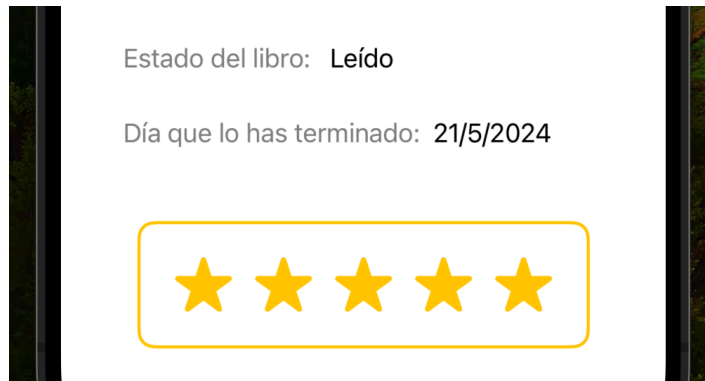
Esta vista la utilizamos para hacer que en la imagen de las estrellas se pueda seleccionar la cantidad de estrellitas y convertir esto en la puntuación que le hayamos dado al libro.



Esto se visualiza en AddBookView de la siguiente forma:



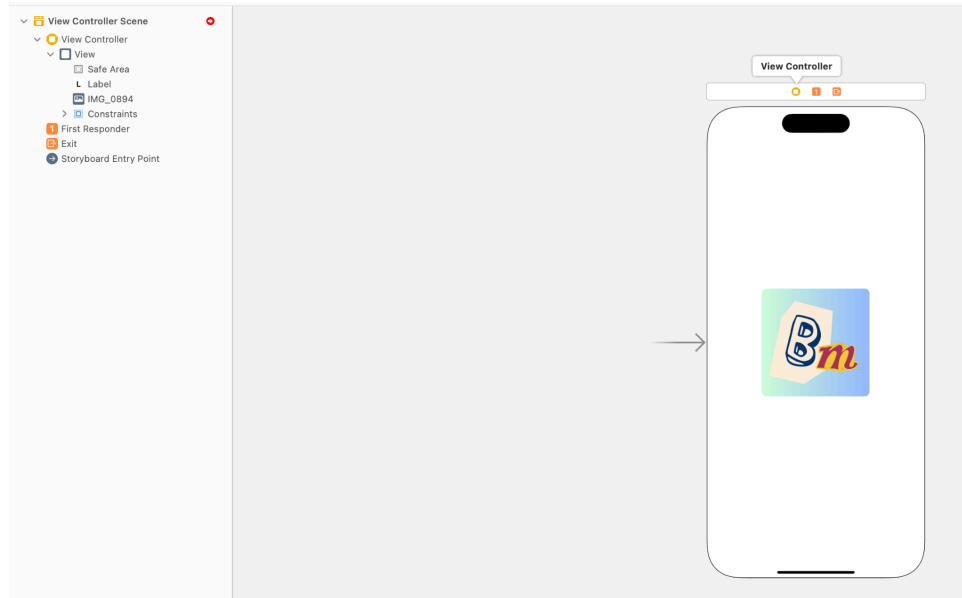
En DetailView se visualiza de la siguiente manera:



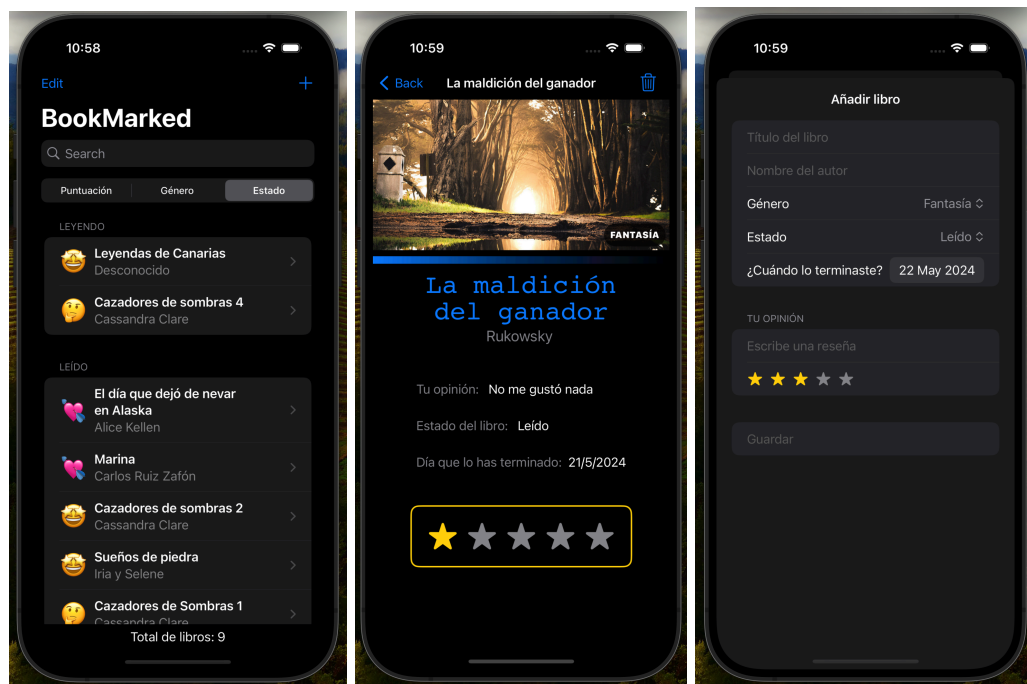
4.7. *ScreenLaunch* o *SplashScreen*:

Antes de poder acceder a la aplicación he agregado un launch screen en el que aparece un momento el icono de la aplicación y después aparece la vista principal de la aplicación, para esto agregué la vista que ya tiene predefinida SwiftUI y la configuré como yo la necesitaba añadiendo el icono de la aplicación en el centro y ajustando los tamaños.

La configuración de la pantalla.



4.8. Vista de la aplicación en modo oscuro:



5. Uso de SwiftData.

Al ejecutar la aplicación podemos rellenar la lista con distintos libros y al cerrarla y volver a abrirla esos libros continuarán ahí con su información, esto es porque la información se guarda a nivel local, lo que significa que la información se guardará en el

dispositivo donde ejecuto la aplicación, esto pasa gracias a SwiftData, y, ¿qué es SwiftData? SwiftData es un nuevo Framework de Apple que nos permite tener bases de datos dentro de nuestra aplicación desarrollada para trabajar con bases de datos en la plataforma Swift, este Framework nos permite crear una base de datos dentro de nuestra app y proporciona herramientas que facilitan la gestión de datos (creación, consulta, actualización y eliminación de registros en una base de datos), está pensado completamente para SwiftUI pero se puede desacoplar de la UI, en el momento en el que desinstalamos la aplicación, toda la información que tuviésemos en ella desaparece para siempre (al utilizar SwiftData, antes de desinstalar la aplicación nos salta una advertencia con que vamos a perder todos los datos que haya en la aplicación).



SwiftData lo utilizamos en las vistas de ContentView y DetailView que son las dos vistas donde necesitamos mostrar información guardada, para poder utilizarlo primero tendremos que importarlo.

6. Problemas que han surgido y soluciones.

Uno de los principales problemas que me surgieron fue que añadí CurrentData como nuevo campo a la base de datos, hice la referencia en todos los lugares donde lo necesitaba pero entonces la aplicación me dejó de funcionar por completo, esto era porque al añadir un nuevo campo a la base de datos hay que desinstalar la aplicación y volver a instalarla para que SwiftData ejecute los cambios.

Otro de los problemas que me surgieron fue la validación del formulario y es que a veces se me activaba el botón de guardar y otras veces no, esto era porque estaba llamando mal

a la función que realizaba la validación, entonces corregí esa línea de código y ya funcionaba perfectamente.

También me ocurrió que en la vista principal (ContentView) se me eliminaban los libros de forma incorrecta cuando deslizabas sobre ellos, esto era debido a que la función DeleteBooks se estaba ejecutando mal, corregí la función y pasó a funcionar todo correctamente.

Al añadir el DatePicker automáticamente aparecerá con la hora actual y el día actual, pero yo sólo quería que apareciese el día y poder elegir la fecha en la que se terminó la lectura sobre un calendario, esto tuvo una solución muy fácil, en la línea 44 de AddBookView había que añadir una modificación a DatePicker, y así de esta forma sólo aparecerá la fecha y no la hora.

```
43  
44         DatePicker("¿Cuándo lo terminaste?", selection: $currentDate, in: ...today, displayedComponents: .date)  
45     }
```

El último error que me dio el programa fue que no me mostraban las imágenes del género en cada uno de los libros, esto era porque en Assets las imágenes tenían el nombre en inglés y luego yo en el código hacía referencia a ellos en castellano, la solución fue llamar de una misma forma a las imágenes en ambos lugares y se solucionó el problema.

7. Fase de pruebas.

En la fase de pruebas he comprobado que todo funcionase correctamente. Lo que más me preocupaba era que los libros se guardarán de forma correcta y se filtraran en la lista de la vista principal según la puntuación, el género y el estado que tuviesen, esto no me dio ningún problema todo se guardaba de forma correcta gracias a *SwiftData*, pero con estas pruebas sí me di cuenta que la imagen del *DetailView* del género al que pertenecen los libros no se visualizaba y esto era debido a que en Assets las tenía nombradas de una forma y en *DetailView* las llamaba de otra forma. Al realizar más pruebas me di cuenta que al visualizar la vista *DetailView* si el estado del libro era “Leyendo” se veía la fecha actual en la que habíamos añadido la lectura como si fuese la fecha en la que habíamos

terminado el libro, para solucionar esto tuve que realizar varios cambios en el código ya que lo primero que había que solucionar era la vista de AddBookView que era donde surgía todo el problema ya que siempre guardaba una fecha entonces aquí añadí un condicional en el que a veces guarda la fecha y otras veces no, lo siguiente fue volver a la vista de DetailView y corregir el código para que cuando el estado del libro fuese “Leyendo” no se mostrase nada respecto a la fecha.

Batería de pruebas que he realizado:

- Comprobar que los libros se guardan correctamente
- Comprobar que se filtra correctamente el picker de la vista principal según la puntuación, género y estado de la lectura.
- Comprobar que el buscador busque por el título del libro o por el autor.
- Comprobar que los emojis que salen en la vista principal junto al título del libro sean los que corresponde a la puntuación del libro.
- Comprobar que en el DetailView de los libros con estado “Leyendo” no se visualice ninguna fecha y que en cambio en los libros con estado “Leído” sí que se visualice la fecha de finalización de la lectura.
- Comprobar que en la vista DetailView se visualice correctamente la imagen que le corresponda según el género del que sea el libro.
- Comprobar que en ContentView se eliminan correctamente los libros de la lista.
- Comprobar que en la vista AddBookView valida que si no están todos los campos rellenados no se puede guardar la lectura.
- Comprobar que en la vista AddBookView al elegir el estado “Leyendo” no nos deje seleccionar ninguna fecha.
- Comprobar que en cada una de las secciones de las distintas categorías del picker de ContentView los libros están ordenados de mayor a menor puntuación.

8. Conclusión.

- **Sí cumple los objetivos.**

La aplicación cumple con todos los objetivos que yo tenía inicialmente cuando se me ocurrió la idea, añade libros a una lista que los clasifica según la puntuación, el género o el estado que yo le haya dado, además puedes eliminar los libros de la lista y es completamente personal.

- **Propuesta de modificaciones.**

Una implementación interesante de realizar sería que en esta misma aplicación tuvieses otra nueva vista donde añadir las películas y series que hayas visto y poder rellenar un pequeño formulario al igual que con los libros, después estas se añadirían a otra pestaña similar a la lista de libros pero con sólo películas o series.

Una buena implementación sería poder editar las reseñas ya escritas por si nos hemos equivocado o nuestra opinión ha cambiado, esta sería una de las primeras mejoras que se realizarían.

Y por último una mejora muy buena sería que la aplicación recomendase lecturas que podrían gustarte en base a los libros que has leído (esto se haría con inteligencia artificial y machine learning).

9. Documentación del sistema desarrollado.

- Manual de uso

Para poder utilizar esta aplicación será necesario tener una versión de iOS a partir de iOS 17.

Funcionalidades principales: añadir libros a nuestra lista personal de lecturas, ver los detalles de esos libros, eliminar los libros de la lista.

Para instalar la aplicación en nuestro dispositivo móvil tendremos que acceder a la AppStore y ahí buscar la aplicación por su nombre 'BookMarked', una vez la encontremos hay que clicar en descargar y a continuación podremos ver que la tenemos disponible en nuestro dispositivo Apple.

Flujo de la aplicación: al abrir la aplicación por primera vez veremos que accedemos a una pantalla en la que sólo aparece el nombre de la aplicación, un símbolo '+' en la parte superior, un buscador y un picker; lo primero que haremos será añadir nuestra primera lectura pulsando en ese símbolo '+', al pulsar ahí veremos que nos sale una nueva vista en la que tenemos un formulario que nos pide los datos del libro que queremos añadir a la lista, rellenamos todos los campos del formulario y le damos a guardar, la aplicación nos devuelve de nuevo a la pantalla principal y ahí podremos ver el libro que hayamos añadido dentro de una categoría y con un emoji que le corresponde según la puntuación que le hayamos dado a la lectura; si clicamos sobre el libro accedemos a una nueva vista en la que podemos ver los detalles del libro y nuestra opinión sobre ellos; si volvemos a la pantalla principal podemos probar a filtrar con el picker el libro que hayamos añadido, lo podemos filtrar por la puntuación que le hayamos dado, por su género y por el estado de la lectura (si lo estamos leyendo o si ya lo hemos leído). Si queremos eliminar alguna de las lecturas podemos hacerlo de tres formas distintas: desde la vista principal deslizando el dedo hacia la izquierda, también desde la vista principal en la parte superior clicamos sobre 'edit' y podemos seleccionar las lecturas que queramos borrar, y por último dentro de la vista detallada de cada uno de los libros en la parte superior tenemos el símbolo de una papelera en la que si clicamos nos sale una advertencia sobre que vamos a eliminar ese libro.

Solución de problemas: para un correcto uso y funcionamiento de la aplicación rellena correctamente todos los campos del formulario para poder guardar los libros sin ningún problema.

Información de contacto: para cualquier problema con la aplicación puede ponerse en contacto con el desarrollador iroruwembley@gmail.com

10. Bibliografía

- AppleDevelopers: <https://developer.apple.com/>
- FlowStack: <https://flowstack.com/>
- SwiftBeta: <https://www.swiftbeta.com/>
- YouTube: <https://www.youtube.com/>
- StackOverFlow: <https://stackoverflow.com>
- Apple Forum Developer: <https://forums.developer.apple.com>

Anexos

Anexo 1. Código de *ContentView* (página principal)

```

//
// ContentView.swift
// Books
//
// Created by Isabel Romero on 18/4/24.
//
import SwiftUI
import SwiftData
struct ContentView: View {
    @Environment(\.modelContext) var modelContext
    @Query(sort: [
        SortDescriptor(\Book.title),
        SortDescriptor(\Book.author)
    ]) var books: [Book]
    @State private var showingAddScreen = false

    @State private var criteria = ""

    @State private var currentTab = 0

    @State private var selectedGenre = "Fantasía"

    @State private var selectedFilter = "Puntuación"

    @State private var searchText = ""

    // @State private var isSearchBarVisible = false

    @State private var searchIsActive = false

    @Environment(\.dismissSearch) private var dismissSearch

    func filteredBooks() -> [Book]{
        if searchText.isEmpty { //si el texto está vacío, mostrar todos los libros
            return books
        } else {
            let filtered = books.filter { $0.title.localizedCaseInsensitiveContains(searchText) ||
$0.author.localizedCaseInsensitiveContains(searchText)}
            print("Filtered books count: \(filtered.count)")
            return filtered
        }
    }
}

```

```

    }
}

var body: some View {
    NavigationStack{
        Picker("Filtrar por:", selection: $selectedFilter) {
            Text("Puntuación").tag("Puntuación")
            Text("Género").tag("Género")
            Text("Estado").tag("Estado")
        }
        .pickerStyle(SegmentedPickerStyle())
        .padding(.horizontal)

        List{
            if selectedFilter == "Puntuación" {
                if !filteredBooks().filter({ $0.rating == 5 }).isEmpty {
                    Section("Mis favoritos") {
                        ForEach(filteredBooks().filter({ $0.rating == 5 })) { book in
                            NavigationLink(value: book){
                                HStack{
                                    EmojiRatingView(rating: book.rating)
                                    .font(.largeTitle)
                                    VStack(alignment: .leading){
                                        Text(book.title)
                                        .font(.headline)
                                        Text(book.author)
                                        .foregroundColor(.secondary)
                                    }
                                }
                            }
                        }
                    }
                }
                .onDelete(perform: deleteBooks)
            }

            if !filteredBooks().filter({ $0.rating == 4 }).isEmpty {
                Section("Me gustaron") {
                    ForEach(filteredBooks().filter({ $0.rating == 4 })) { book in
                        NavigationLink(value: book){
                            HStack{
                                EmojiRatingView(rating: book.rating)
                                .font(.largeTitle)

```

```

        VStack(alignment: .leading){
            Text(book.title)
                .font(.headline)
            Text(book.author)
                .foregroundColor(.secondary)
        }
    }
}
}
.onDelete(perform: deleteBooks)
}
}

if !filteredBooks().filter({ $0.rating == 3 }).isEmpty {
    Section("Fue normalito") {
        ForEach(filteredBooks().filter({ $0.rating == 3 })) { book in
            NavigationLink(value: book){
                HStack{
                    EmojiRatingView(rating: book.rating)
                        .font(.largeTitle)
                    VStack(alignment: .leading){
                        Text(book.title)
                            .font(.headline)
                        Text(book.author)
                            .foregroundColor(.secondary)
                    }
                }
            }
        }
    }
    .onDelete(perform: deleteBooks)
}

if !filteredBooks().filter({ $0.rating == 2 }).isEmpty {
    Section("Meh") {
        ForEach(filteredBooks().filter({ $0.rating == 2 })) { book in
            NavigationLink(value: book){
                HStack{
                    EmojiRatingView(rating: book.rating)
                        .font(.largeTitle)
                    VStack(alignment: .leading){
                        Text(book.title)

```

```

        .font(.headline)
        Text(book.author)
        .foregroundColor(.secondary)
    }
}
}
}
.onDelete(perform: deleteBooks)
}
}

if !filteredBooks().filter({ $0.rating == 1 }).isEmpty {
    Section("He perdido el tiempo") {
        ForEach(filteredBooks().filter({ $0.rating == 1 })) { book in
            NavigationLink(value: book){
                HStack{
                    EmojiRatingView(rating: book.rating)
                    .font(.largeTitle)
                    VStack(alignment: .leading){
                        Text(book.title)
                        .font(.headline)
                        Text(book.author)
                        .foregroundColor(.secondary)
                    }
                }
            }
        }
        .onDelete(perform: deleteBooks)
    }
}

else if selectedFilter == "Género" {
    if !filteredBooks().filter({ $0.genre == "Fantasía" }).isEmpty {
        Section("Fantasía") {
            ForEach(filteredBooks().filter({ $0.genre == "Fantasía" }).sorted(by: { $0.rating > $1.rating })) { book in
                NavigationLink(value: book){
                    HStack{
                        EmojiRatingView(rating: book.rating)
                        .font(.largeTitle)
                        VStack(alignment: .leading){
                            Text(book.title)

```

```

        .font(.headline)
        Text(book.author)
        .foregroundColor(.secondary)
    }
}
}
}
.onDelete(perform: deleteBooks)
}
}

if !filteredBooks().filter({ $0.genre == "Horror" }).isEmpty {
    Section("Horror") {
        ForEach(filteredBooks().filter({ $0.genre == "Horror" }).sorted(by: { $0.rating > $1.rating })) { book in
            NavigationLink(value: book){
                HStack{
                    EmojiRatingView(rating: book.rating)
                    .font(.largeTitle)
                    VStack(alignment: .leading){
                        Text(book.title)
                            .font(.headline)
                        Text(book.author)
                            .foregroundColor(.secondary)
                    }
                }
            }
        }
        .onDelete(perform: deleteBooks)
    }
}

if !filteredBooks().filter({ $0.genre == "Infantil" }).isEmpty {
    Section("Infantil") {
        ForEach(filteredBooks().filter({ $0.genre == "Infantil" }).sorted(by: { $0.rating > $1.rating })) { book in
            NavigationLink(value: book){
                HStack{
                    EmojiRatingView(rating: book.rating)
                    .font(.largeTitle)
                    VStack(alignment: .leading){
                        Text(book.title)
                            .font(.headline)
                        Text(book.author)
                    }
                }
            }
        }
    }
}

```

```

        .foregroundStyle(.secondary)
    }
}
}
}
}
.delete(onDelete: perform: deleteBooks)
}
}

if !filteredBooks().filter({ $0.genre == "Misterio" }).isEmpty {
    Section("Misterio") {
        ForEach(filteredBooks().filter({ $0.genre == "Misterio" }).sorted(by: { $0.rating > $1.rating })) { book in
            NavigationLink(value: book){
                HStack{
                    EmojiRatingView(rating: book.rating)
                    .font(.largeTitle)
                    VStack(alignment: .leading){
                        Text(book.title)
                            .font(.headline)
                        Text(book.author)
                            .foregroundStyle(.secondary)
                    }
                }
            }
        }
        .onDelete(perform: deleteBooks)
    }
}

if !filteredBooks().filter({ $0.genre == "Poemario" }).isEmpty {
    Section("Poemario") {
        ForEach(filteredBooks().filter({ $0.genre == "Poemario" }).sorted(by: { $0.rating > $1.rating })) { book in
            NavigationLink(value: book){
                HStack{
                    EmojiRatingView(rating: book.rating)
                    .font(.largeTitle)
                    VStack(alignment: .leading){
                        Text(book.title)
                            .font(.headline)
                        Text(book.author)
                            .foregroundStyle(.secondary)
                    }
                }
            }
        }
    }
}

```

```

        }
    }
}
.onDelete(perform: deleteBooks)
}
}

if !filteredBooks().filter({ $0.genre == "Romance" }).isEmpty {
    Section("Romance") {
        ForEach(filteredBooks().filter({ $0.genre == "Romance" }).sorted(by: { $0.rating > $1.rating })) { book in
            NavigationLink(value: book){
                HStack{
                    EmojiRatingView(rating: book.rating)
                        .font(.largeTitle)
                    VStack(alignment: .leading){
                        Text(book.title)
                            .font(.headline)
                        Text(book.author)
                            .foregroundColor(.secondary)
                    }
                }
            }
        }
    }
    .onDelete(perform: deleteBooks)
}

if !filteredBooks().filter({ $0.genre == "Thriller" }).isEmpty {
    Section("Thriller") {
        ForEach(filteredBooks().filter({ $0.genre == "Thriller" }).sorted(by: { $0.rating > $1.rating })) { book in
            NavigationLink(value: book){
                HStack{
                    EmojiRatingView(rating: book.rating)
                        .font(.largeTitle)
                    VStack(alignment: .leading){
                        Text(book.title)
                            .font(.headline)
                        Text(book.author)
                            .foregroundColor(.secondary)
                    }
                }
            }
        }
    }
}

```

```

    }
    .onDelete(perform: deleteBooks)
  }
}

} else if selectedFilter == "Estado" {
  if !filteredBooks().filter({ $0.moment == "Leyendo" }).isEmpty {
    Section("Leyendo") {
      ForEach(filteredBooks().filter({ $0.moment == "Leyendo" }).sorted(by: { $0.rating > $1.rating }))) { book in
        NavigationLink(value: book){
          HStack{
            EmojiRatingView(rating: book.rating)
            .font(.largeTitle)
            VStack(alignment: .leading){
              Text(book.title)
                .font(.headline)
              Text(book.author)
                .foregroundColor(.secondary)
            }
          }
        }
      }
    }
    .onDelete(perform: deleteBooks)
  }
}

if !filteredBooks().filter({ $0.moment == "Leído" }).isEmpty {
  Section("Leído") {
    ForEach(filteredBooks().filter({ $0.moment == "Leído" }).sorted(by: { $0.rating > $1.rating }))) { book in
      NavigationLink(value: book){
        HStack{
          EmojiRatingView(rating: book.rating)
            .font(.largeTitle)
          VStack(alignment: .leading){
            Text(book.title)
              .font(.headline)
            Text(book.author)
              .foregroundColor(.secondary)
          }
        }
      }
    }
  }
  .onDelete(perform: deleteBooks)
}

```

```

        }
    }
}

.navigationDestination(for: Book.self){ book in
    DetailView(book: book)
}

Text("Total de libros: \${books.count}")
.navigationTitle("BookMarked")
.toolbar{
    ToolbarItem(placement: .topBarLeading){
        EditButton()
    }
    ToolbarItem(placement: .topBarTrailing){
        Button("Add book", systemImage: "plus"){
            showingAddScreen.toggle()
        }
    }
}

.sheet(isPresented: $showingAddScreen){
    AddBookView()
}

.scrollContentBackground(.hidden)
}

.searchable(text: $searchText, isPresented: $searchIsActive)
}

func deleteBooks(at offsets: IndexSet){
    for offset in offsets{
        //find this book in our query
        let book = books[offset]

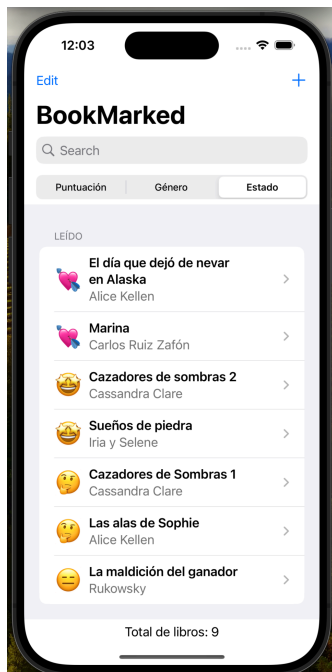
        //delete it from the context
        modelContext.delete(book)
    }
}

}

#Preview {
    ContentView()
}

```

Vista:



Anexo 2. Código *DetailView*

```
//
// DetailView.swift
// Books
//
// Created by Isabel Romero on 19/4/24.
//

import SwiftUI
import SwiftData

struct DetailView: View {

    @Environment(\.modelContext) var modelContext
    @Environment(\.dismiss) var dismiss
    @State private var showingDeleteAlert = false
    @State private var dragAmount = CGSize.zero

    let book: Book

    var body: some View {
```

```
ScrollView{
    ZStack(alignment: .bottomTrailing){
        Image(book.genre)
            .resizable()
            .scaledToFit()
        Text(book.genre.uppercased())
            .font(.caption)
            .fontWeight(.black)
            .padding(8)
            .foregroundColor(.white)
            .background(.black.opacity(0.75))
            .clipShape(.capsule)
            .offset(x: -5, y: -5)
    }
    LinearGradient(colors: [.blue, .black], startPoint: .topLeading, endPoint: .bottomTrailing)
    Text(book.title)
        .font(.custom("Courier", size: 35))
//    .foregroundColor(.secondary)
    .foregroundColor(.blue)
    .multilineTextAlignment(.center)
    .padding(.horizontal, 20)

    Text(book.author)
        .font(.title3)
        .foregroundColor(.secondary)
        .padding(.bottom, 30)

    VStack(alignment: .leading){
        HStack {
            Text("Tu opinión: ")
                .foregroundColor(.secondary)

            Text(book.review)
        }
        .padding(10)
        .padding(.horizontal, 30)

        HStack {
            Text("Estado del libro: ")
                .foregroundColor(.secondary)
            Text(book.moment)
        }
    }
```

```

        .padding(10)
        .padding(.horizontal, 30)

        HStack{
            if let currentDateString = book.currentDateAsString() {
                HStack{
                    Text("Día que lo has terminado:")
                        .foregroundColor(.secondary)
                    Text(currentDateString)
                }

            } else {
                Text("")
            }
        }
        .padding(10)
        .padding(.horizontal, 30)

    }
    RatingView(rating: .constant(book.rating))
        .font(.largeTitle)
        .padding(20)
        .background(
            RoundedRectangle(cornerRadius: 10)
                .stroke(Color.yellow, lineWidth: 2)
        )
        .padding(30)

    }
    .navigationTitle(book.title)
    .navigationBarTitleDisplayMode(.inline)
    .scrollBounceBehavior(.basedOnSize)
    .alert("Delete book", isPresented: $showingDeleteAlert){
        Button("Delete", role: .destructive, action: deleteBook)
        Button("Cancel", role: .cancel){ }
    }message: {
        Text("Are you sure?")
    }
    .toolbar{
        Button("Delete this book", systemImage:"trash"){
            showingDeleteAlert = true
        }
    }

```

```

    }

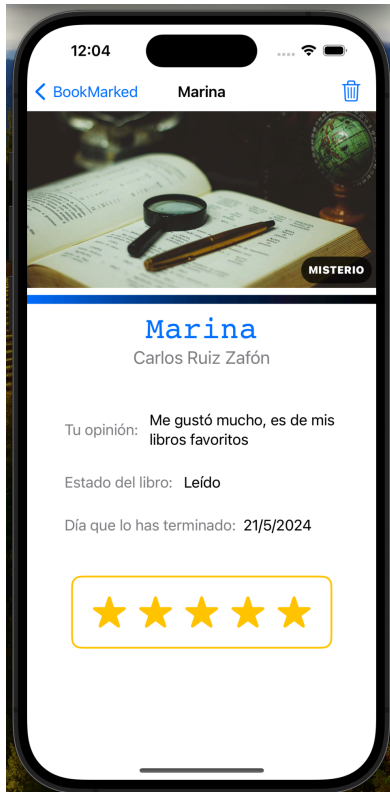
//    .toolbar{
//        Button(action: {
//            isEditing.toggle()
//        }) {
//            Image(systemName: isEditing ? "pencil.slash" : "pencil")
//        }
//        .foregroundColor(.primary)
//    }
}

func deleteBook(){
    modelContext.delete(book)
    dismiss()
}
}

#Preview {
    do{
        let config = ModelConfiguration(isStoredInMemoryOnly: true)
        let container = try ModelContainer(for: Book.self, configurations: config)
        let example = Book(title: "Test book", author: "Test author", genre: "Fantasy", review: "Great book", rating: 4,
saveCurrentDate: true, moment: "Leído")
        return DetailView(book: example)
            .modelContainer(container)
    }catch{
        return Text("Failed to create preview: \(error.localizedDescription)")
    }
}
}

```

Vista:



Anexo 3. Código *AddBookView*:

```
//
// AddBookView.swift
// Books
//
// Created by Isabel Romero on 18/4/24.
//
import SwiftUI
struct AddBookView: View {

    @Environment(\.modelContext) var modelContext
    @Environment(\.dismiss) var dismiss
    @State private var title = ""
    @State private var author = ""
    @State private var rating = 3
    @State private var genre = "Fantasía"
    @State private var review = ""
    // @State private var currentDate = Date()
    @State private var moment = "Leído"
```

```

    let bookWithDate = Book(title: "Title", author: "Author", genre: "Genre", review: "Review", rating: 5, saveCurrentDate: true,
moment: "Moment") // Guarda la fecha actual

    let bookWithoutDate = Book(title: "Title", author: "Author", genre: "Genre", review: "Review", rating: 5, saveCurrentDate:
false, moment: "Moment") // No guarda la fecha

    let today = Date()

    @State var isPresented: Bool = false

    var disableForm: Bool{
        title.count < 1 || author.count < 1 || review.count < 1
    }

    let genres = ["Fantasía", "Horror", "Infantil", "Misterio", "Poemario", "Romance", "Thriller"]

    let moments = ["Leído", "Leyendo"]

    var body: some View {
        NavigationStack{
            Form{
                Section{
                    TextField("Título del libro", text: $title)
                    TextField("Nombre del autor", text: $author)

                    Picker("Género",selection: $genre){
                        ForEach(genres, id: \.self){
                            Text($0)
                        }
                    }

                    Picker("Estado",selection: $moment){
                        ForEach(moments, id: \.self){
                            Text($0)
                        }
                    }

                    VStack {
                        if moment == "Leído" {
                            DatePicker("¿Cuándo lo terminaste?", selection: .constant(Date()), displayedComponents: .date)
                        } else {
                            Text("Lo estás leyendo")
                        }
                    }
                }
            }
        }
    }

```

```

    }

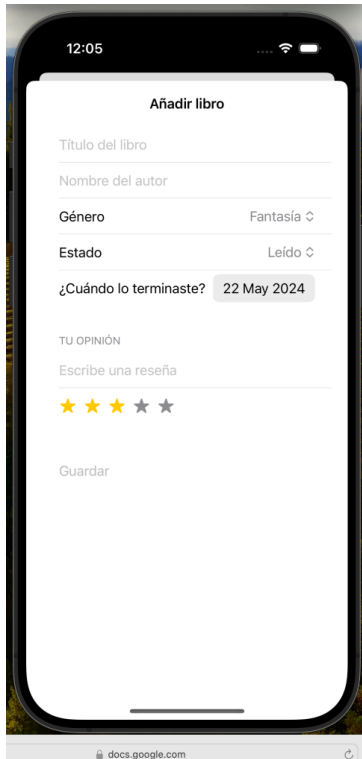
    Section("Tu opinión"){
        TextField("Escribe una reseña", text: $review)
        RatingView(rating: $rating)
    }

    Section{
        Button("Guardar") {
            let newBook: Book
            if moment == "Leído" {
                newBook = Book(title: title, author: author, genre: genre, review: review, rating: rating, saveCurrentDate:
true, moment: moment)
            } else {
                newBook = Book(title: title, author: author, genre: genre, review: review, rating: rating, saveCurrentDate:
false, moment: moment)
            }
            modelContext.insert(newBook)
            isPresented = true
            dismiss()
        }
    }
    .disabled(disableForm)
}
.navigationTitle("Añadir libro")
}
}

#Preview {
    AddBookView()
}

```

Vista:



Anexo 4. Código de *RatingView*:

```
//
// RatingView.swift
// Books
//
// Created by Isabel Romero on 18/4/24.
//
import SwiftUI
struct RatingView: View {

    @Binding var rating: Int

    var label = ""

    var maximumRating = 5

    var offImage: Image?
    var onImage = Image(systemName: "star.fill")

    var offColor = Color.gray
    var onColor = Color.yellow
```

```

var body: some View {
    HStack{
        if label.isEmpty == false {
            Text(label)
        }
        ForEach(1..<maximumRating + 1, id: \.self){number in
            Button{
                rating = number
            } label: {
                image(for: number)
                    .foregroundColor(number > rating ? offColor : onColor)
            }
        }
    }
    .buttonStyle(.plain)
}

func image (for number: Int) -> Image{
    if number > rating{
        offImage ?? onImage
    }else{
        onImage
    }
}

#Preview {
    RatingView(rating: .constant(1))
}

```

Vista:



Anexo 5. Código *EmojiRatingView* (a esta vista se la llama en el ContentView y depende de la puntuación que tenga el libro tendrá uno u otro emoji).

```
//  
// EmojiRatingView.swift  
// Books  
//  
// Created by Isabel Romero on 19/4/24.  
//  
import SwiftUI  
struct EmojiRatingView: View {  
    let rating: Int  
  
    var body: some View {  
        switch rating {  
  
            case 1:  
                Text("😞")  
  
            case 2:  
                Text("😐")  
            case 3:  
                Text("😓")  
            case 4:  
                Text("😄")  
            default:  
                Text("❤️")  
        }  
    }  
}  
  
#Preview {  
    EmojiRatingView(rating: 3)  
}
```

Vista:



Anexo 6. Clase *book*:

```
// Book.swift
// Books
//
// Created by Isabel Romero on 18/4/24.
//
import Foundation
import SwiftData
@Model
class Book {
    var title: String
    var author: String
    var genre: String
    var review: String
    var rating: Int
    var currentDate: Date?
    var moment: String

    init(title: String, author: String, genre: String, review: String, rating: Int, saveCurrentDate: Bool = false, moment: String) {
        self.title = title
        self.author = author
        self.genre = genre
        self.review = review
        self.rating = rating
        self.moment = moment

        if saveCurrentDate {
            self.currentDate = Date()
        } else {
            self.currentDate = nil
        }
    }
}
```

```
func currentDateAsString() -> String? {  
    guard let date = currentDate else { return nil }  
    let dateFormatter = DateFormatter()  
    dateFormatter.dateFormat = "d/M/yyyy"  
    return dateFormatter.string(from: date)  
}  
}
```